Atelier pratique n°3 — Les bases de Git

Corrigé des questions et bilan des commandes

Objectif : comprendre la création d'un dépôt Git, le suivi des fichiers, les commits et la consultation de l'historique.

PARTIE 1 Configuration et création du projet

À quoi sert la commande `git init` ?
Elle initialise un nouveau dépôt Git dans le dossier courant. Git crée le dossier caché `.git`, qui contient les informations nécessaires au suivi de l'historique du projet.

Quel est le rôle des commandes `git config` ?
Elles permettent de configurer Git. Par exemple, `git config --global user.name` définit le nom de l'utilisateur et `git config --global user.email` définit son adresse e-mail. L'option `--global` applique ces paramètres à tous les dépôts de l'utilisateur.

Que signifie l'expression *working directory* ?
Le *working directory* (répertoire de travail) est le dossier contenant les fichiers du projet sur lesquels on travaille et que l'on peut modifier avant de préparer ou d'enregistrer ces changements dans Git.

PARTIE 2 Suivi des fichiers et premier commit

Quelle est la différence entre `git add` et `git commit` ?
`git add` prépare les modifications en les plaçant dans la zone de staging. `git commit` enregistre un instantané des modifications préparées dans l'historique du dépôt.

Quel est le rôle de la zone de staging ?
La zone de staging permet de sélectionner précisément les modifications qui seront incluses dans le prochain commit. On peut ainsi préparer un ensemble cohérent de changements.

À quoi sert l'option `-m` ?
Dans `git commit -m "message"`, l'option `-m` permet d'écrire directement le message du commit dans la commande, sans ouvrir un éditeur de texte.

PARTIE 3 Modifier un fichier

Que montre `git diff` ?
La commande `git diff` affiche les différences entre le répertoire de travail et la zone de staging : elle montre les modifications qui n'ont pas encore été ajoutées avec `git add`.

Atelier pratique n°3 — Les bases de Git

Corrigé des questions et bilan des commandes

Quelle est la différence entre `git diff` et `git diff --staged` ?
`git diff` affiche les modifications non préparées entre le répertoire de travail et la zone de staging. `git diff --staged` affiche les modifications déjà préparées dans la zone de staging par rapport au dernier commit.

Pourquoi créer plusieurs commits au lieu d'un seul ?
Plusieurs commits bien nommés permettent de séparer les changements par sujet, de mieux comprendre l'historique, de retrouver plus facilement une modification et de faciliter la collaboration ou le dépannage.

PARTIE 4 Gérer plusieurs fichiers

Quelle est la différence entre un fichier *tracked* et *untracked* ?
Un fichier *tracked* (suivi) est connu de Git parce qu'il a déjà été ajouté au suivi ou enregistré dans un commit. Un fichier *untracked* (non suivi) se trouve dans le dossier du projet, mais Git ne le suit pas encore.

Peut-on ajouter plusieurs fichiers avec une seule commande `git add` ?
Oui. Par exemple, `git add style.css README.md` prépare les deux fichiers dans une seule commande. On peut aussi utiliser `git add .` pour préparer les changements du dossier courant, en vérifiant auparavant ce que l'on souhaite inclure.

À quoi sert le fichier `README.md` ?
Il présente le projet : son objectif, son contenu, les instructions d'installation ou d'utilisation et toute information utile aux personnes qui consultent le dépôt.

PARTIE 5 Bilan et vérification

Tableau complété demandé dans l'atelier

Commande	Rôle
`git init`	Initialise un dépôt Git dans le dossier courant.
`git status`	Affiche l'état du dépôt : fichiers modifiés, préparés, non suivis et état de la branche.
`git add`	Ajoute des fichiers ou des modifications à la zone de staging pour préparer le prochain commit.
`git commit`	Enregistre les modifications préparées dans l'historique du dépôt avec un message.
`git log`	Affiche l'historique des commits du dépôt.

Commande	Rôle
`git diff`	Affiche les différences entre les fichiers du répertoire de travail et la zone de staging.

Récapitulatif des commandes principales

Commande	Utilité
`git --version`	Vérifier la version de Git installée.
`git config --global user.name "Votre Nom"`	Définir le nom associé aux commits.
`git config --global user.email "votre@email.com"`	Définir l'adresse e-mail associée aux commits.
`mkdir atelier-git`	Créer le dossier du projet.
`cd atelier-git`	Entrer dans le dossier du projet.
`git init`	Initialiser le dépôt.
`git status`	Vérifier l'état des fichiers.
`git add index.html`	Préparer index.html.
`git commit -m "Initialisation du projet"`	Créer le premier commit.
`git diff`	Voir les modifications non préparées.
`git diff --staged`	Voir les modifications préparées.
`git add style.css README.md`	Préparer deux fichiers en une commande.
`git log --oneline`	Afficher l'historique sous forme compacte.

Livrable attendu — vérification

- Le dossier `atelier-git` contient `index.html`, `style.css` et `README.md`.
- Le dépôt Git local est initialisé.
- Au moins trois commits ont été créés avec des messages explicites.
- Une capture d'écran de la commande `git log --oneline` est disponible.